Université Saint-Joseph de Beyrouth
École Supérieure d'Ingénieurs de Beyrouth (ESIB)

https://github.com/Lea721/2d_heat_solver

Parallel 2D Heat Transfer Simulation Project

Presented by:

Kawsar Zbib

Christina Haber

Fatima Al Rabih

Lea Chamseddine

Computer and Communication Engineering

Presented to:

Dr. Maroun Ayli

Contents

I. Introduction

This project focuses on the simulation of 2D transient heat diffusion using numerical methods and parallel computing. The goal is to solve the heat equation over time while improving performance through domain decomposition and communication between multiple MPI processes.

We first implemented a serial finite-difference solver to compute the temperature evolution on a 2D grid. Then, we extended the program to a parallel version using MPI, where the global domain is divided into smaller blocks, each handled by a different process. Neighboring processes exchange temperature values through halo (ghost) cells, allowing the simulation to remain consistent across the full domain.

The project also compares blocking and non-blocking communication, evaluates the effect of different boundary conditions (Dirichlet, Neumann, nonblocking Dirichlet), and measures the performance through strong and weak scaling tests. Visualization scripts generate heatmaps and plots to analyze physical behavior and the speedup obtained through parallelization.

In addition, we developed visualization tools using Matplotlib, OpenGL, and matplotlib.animation.FuncAnimation to create heatmaps, dynamic animations, and real-time renderings of the simulation. These visualizations help analyze both the physical behavior of the system and the speedup obtained through parallelization.

II. Mathematical Model and Numerical Discretization

The goal of this project is to compute how heat diffuses over time in a two-dimensional domain. Physical behavior is governed by the 2D transient heat conduction equation, which relates the change of temperature with respect to time to the spatial distribution of temperature.

1. Governing Heat Equation

The evolution of temperature $T(x, y, t)$is described by:

$$\frac{\partial T}{\partial t} = \alpha\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right)$$

where:

- $T(x, y, t)$: temperature at point $(x, y)$and time t,
- α: thermal diffusivity of the material (constant),
- $\frac{\partial^2 T}{\partial x^2}$, $\frac{\partial^2 T}{\partial y^2}$: the curvature of the temperature profile in both directions.

2. Spatial Discretization of the Domain

The square domain is divided into a uniform grid of $N \times N$nodes:

$$x_i = i\Delta x \ , y_j = j\Delta y$$

where:

- $i = 0,1, \dots, N-1$(x-direction),
- $j = 0,1, \dots, N-1$(y-direction),
- Δx and Δy are the spatial steps (constant in our project).

Each grid point (i, j)holds a temperature value $T_{i,j}$

3. Finite Difference Approximation of the Second Derivatives

To approximate the PDE on a grid, we replace the continuous second derivatives with central finite differences, which use the value of the point and its four direct neighbors.

Second derivative in the x direction:

$$\frac{\partial^2 T}{\partial x^2}(x_i, y_j) \approx \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta x)^2}$$

Second derivative in the y direction:

$$\frac{\partial^2 T}{\partial y^2}(x_i, y_j) \approx \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta y)^2}$$

4. Explicit Time Discretization (Euler Forward Method)

Time is divided into steps of size Δt. Let $T_{i,j}^n$ denote the temperature at time step n. The time derivative is approximated by:

$$\frac{\partial T}{\partial t} \approx \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t}$$

Substituting the spatial finite differences into the heat equation gives the update rule used in the simulation.

5. Final Update Formula Used in the Solver

Combining all approximations, the numerical scheme becomes:

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta t\, \alpha \left[\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{(\Delta x)^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{(\Delta y)^2} \right]$$

It shows clearly that each point in the grid updates using the four neighbors (left, right, top, bottom), which matches the serial and MPI implementations.

6. Stability Condition

Because we use an explicit numerical scheme, the time step Δt must satisfy a stability requirement to avoid divergence. For a 2D grid, the stability limit is:

$$\Delta t \leq \frac{1}{2\alpha} \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right)^{-1}$$

In the project, we ensured stability by selecting a sufficiently small constant Δt.

7. Boundary Conditions Implemented in the Project

Boundary conditions define how heat interacts with the edges of the computational domain. They are essential to ensuring a realistic physical simulation because they dictate whether heat can leave the domain, enter it, or remain trapped inside. In our project, we implemented two physical boundary conditions Dirichlet and Neumann along with a parallel implementation of Dirichlet for the MPI solver. Each condition produces different physical behavior, numerical treatment, and visual results.

7.1. Dirichlet Boundary Condition

The Dirichlet boundary condition imposes a constant, unchanging temperature along all boundaries of the simulation domain. This means that regardless of how heat evolves inside the domain, the outer edges remain fixed at a prescribed temperature. Physically, this represents a situation where the domain is in contact with an external thermal reservoir—such as a heated or cooled frame—that maintains the temperature at the edges.

$$T_{boundary} = C$$

where C is a constant (defined in the code and kept unchanged).
At each iteration, after computing the interior grid updates, the solver explicitly sets:

- Top boundary:

$$T_{0,j} = C$$

- Bottom boundary:

$$T_{N-1,j} = C$$

- Left boundary:

$$T_{i,0} = C$$

- Right boundary:

$$T_{i,N-1} = C$$

This guarantees that the boundaries remain constant over the entire simulation, creating a stable heat profile.
Under Dirichlet boundaries, heat diffuses inward from (or outward toward) the edges, gradually forming smooth, concentric temperature gradients. The edges appear clearly in heatmaps as constant-color lines.

7.2. Neumann Boundary Condition

The Neumann boundary condition models a domain where no heat can enter or leave through the edges. This simulates insulation. The key idea is that heat flux at the boundary must be zero.

$$\frac{\partial T}{\partial n} = 0$$

Meaning the temperature does not change in the outward normal direction.

Zero heat flux is implemented by copying the adjacent interior cell:\

$$T_{0,j} = T_{1,j}, T_{N-1,j} = T_{N-2,j}, T_{i,0} = T_{i,1}, T_{i,N-1} = T_{i,N-2}$$

This produces reflective behavior where heat remains contained inside the domain.

Unlike Dirichlet boundaries, Neumann boundaries do not force the edges to stay constant. Temperatures near the boundary evolve naturally and may rise or equalize over time.

7.3. Dirichlet vs Neumann Boundary Conditions

The figure illustrates the distinct influence of Dirichlet and Neumann boundary conditions on the final temperature distribution. Under Dirichlet conditions, the edges of the domain are fixed at a constant temperature, which produces the dark uniform frame observed in the plot. As heat diffuses from the center toward these fixed boundaries, it is progressively absorbed, resulting in a gradual decay of temperature near the edges and a more concentrated hot region in the middle.

In contrast, Neumann conditions enforce zero heat flux at the boundaries, meaning that the edges do not lose or gain heat. This insulation effect causes the temperature to remain higher across the domain, especially near the boundaries, since thermal energy is trapped inside. The corresponding temperature profile shows that the Dirichlet case drops sharply at the domain limits, while the Neumann case maintains a broader and smoother curve. Overall, the Dirichlet boundary dissipates heat outward, reducing the total internal energy, whereas the Neumann boundary preserves it, producing a warmer and more uniformly distributed final temperature field.

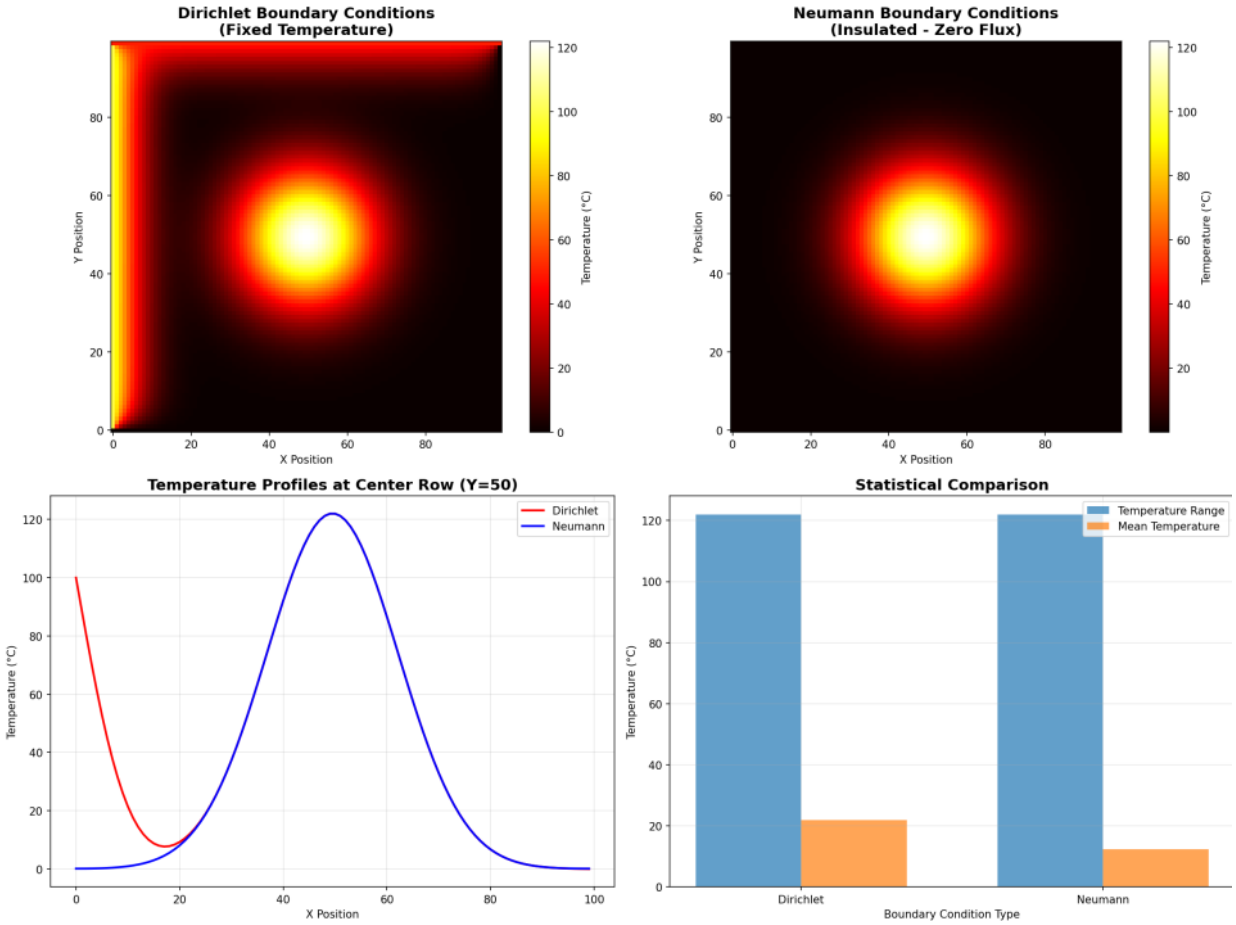

Figure 1- Dirichlet vs Neumann

III. Setting Up Communication Between One Master and Three Worker Nodes

To execute the MPI heat solver across several machines, we deployed the system on a cluster consisting of one master VM and three worker VMs. For reliable communication, all machines were configured on the same network, provided with static IP addresses, and connected through SSH to allow MPI to launch remote processes. The following steps summarize the full procedure we followed to establish communication across the distributed environment.

1. Configure All VMs in Bridged Mode with Static IP Addresses

The first step was to place all VMs in Bridged Networking Mode, ensuring that each machine appears on the local network like a physical computer.

To maintain stable communication and avoid IP changes on reboot, we assigned static IPs to all nodes:
Master: 10.100.100.100
Worker1: 10.100.100.101
Worker2: 10.100.100.102
Worker3: 10.100.100.103
These static IPs ensure that the MPI hostfile always remains valid and that machines can consistently reach each other.
We verified network connectivity:

```
ping 10.100.100.101
ping 10.100.100.102
ping 10.100.100.103
```

Successful replies confirmed that all machines were correctly connected.

2. Test SSH Access Between Master and Workers

MPI relies on SSH to start processes remotely.
We tested connectivity manually:

```
ssh progparallele@10.100.100.101
ssh progparallele@10.100.100.102
ssh progparallele@10.100.100.103
```

This step ensured that communication between nodes was functional.

3. Configure Passwordless SSH Authentication

To allow MPI to start remote processes automatically without password prompts, we enabled passwordless SSH:

```
ssh-keygen -t rsa
ssh-copy-id progparallele@10.100.100.101
ssh-copy-id progparallele@10.100.100.102
ssh-copy-id progparallele@10.100.100.103
```

Verification:

```
ssh progparallele@10.100.100.101
```

This ensured the master could communicate with all workers automatically.

4. Synchronize the Project Files Across All Nodes

Each VM must run the exact same version of the solver.
We copied the entire project from the master to all workers:

```
scp -r ~/Desktop/2d_heat_solver/* progparallele@10.100.100.101:~/Desktop/2d_heat_solver/
```

```
scp -r ~/Desktop/2d_heat_solver/* progparallele@10.100.100.102:~/Desktop/2d_heat_solver/
scp -r ~/Desktop/2d_heat_solver/* progparallele@10.100.100.103:~/Desktop/2d_heat_solver/
```

This ensured all nodes executed the same code.

5. Create the MPI Hostfile

We defined a hostfile listing all nodes and how many MPI processes each should run:

```
10.100.100.100 slots=2
10.100.100.101 slots=2
10.100.100.102 slots=2
10.100.100.103 slots=2
```

Because we used static IPs, this file never needed to change.

6. Run MPI Across All Machines

We launched the solver across the master and three workers:
`mpirun -np 4 --hostfile ~/hosts ~/Desktop/2d_heat_solver/build/heat_solver_mpi`
MPI distributed the processes:

- Rank 0 → Master
- Rank 1 → Worker 1
- Rank 2 → Worker 2
- Rank 3 → Worker 3

7. Verifying Execution on Worker Nodes

Using top

Connect to a worker:
`ssh progparallele@10.100.100.101`
`top`

We see `heat_solver_mpi` running.

Please check the demo video.

IV. Parallel Solver Implementation Using MPI

To improve performance and handle larger grid sizes, the serial heat solver was extended to a parallel version using MPI. MPI allows the domain to be split across multiple processes, distributing the computational load and reducing execution time. The parallel solver

maintains the same numerical method as the serial version but differs in how data is managed, communicated, and updated between subdomains.

1. Domain Decomposition

In the parallel implementation, the global 2D grid is divided into smaller subdomains, each assigned to a different MPI rank. Instead of storing the entire grid, each process holds only a fraction of it, which reduces memory usage and enables simultaneous computation.

This decomposition requires cooperation between processes because the heat update at a given point depends on the values of its four neighbors. When a point lies at the boundary of a subdomain, some of its neighbors belong to another MPI process.

To solve this issue, each process stores an extra layer of halo (ghost) cells. These cells temporarily hold the values received from neighboring processes, allowing each process to apply the update formula independently.

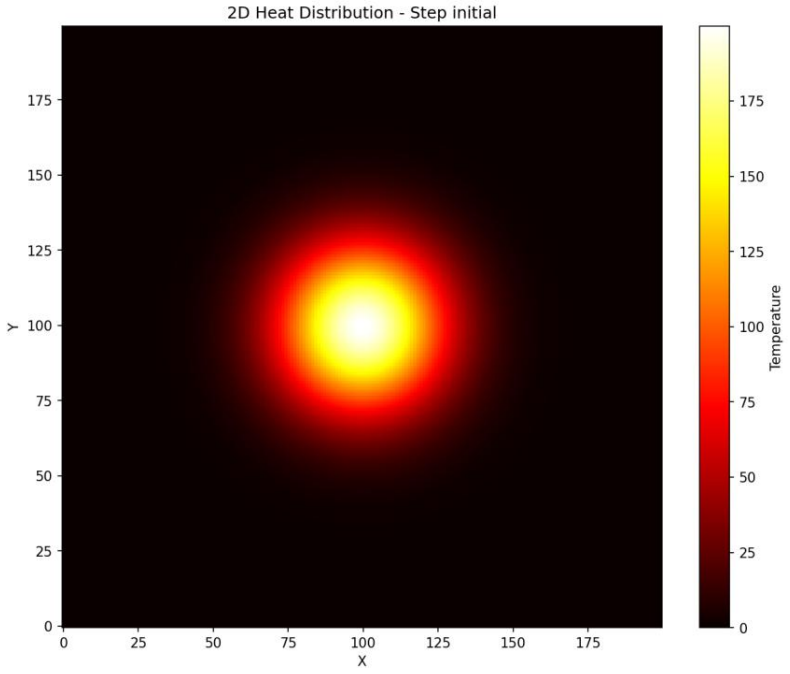

Figure 2- Initial Temperature Distribution

2. Halo Exchange Mechanism

Before each time step, every MPI process exchanges its boundary values with its immediate neighbors:

- Top row ↔ North neighbor
- Bottom row ↔ South neighbor
- Left column ↔ West neighbor
- Right column ↔ East neighbor

After receiving the boundary data, the halo cells are updated, ensuring that all neighboring values are available before computing the heat equation.
This step is crucial because it ensures mathematical consistency between subdomains and allows the parallel solver to reproduce the same results as the serial solver.

3. Blocking Communication (MPI_Sendrecv)

The first parallel version uses blocking communication, where processes wait for messages to be sent and received before continuing. This method is simple and ensures synchronization but can limit performance for large process counts because each process must pause until halo exchange is completed.

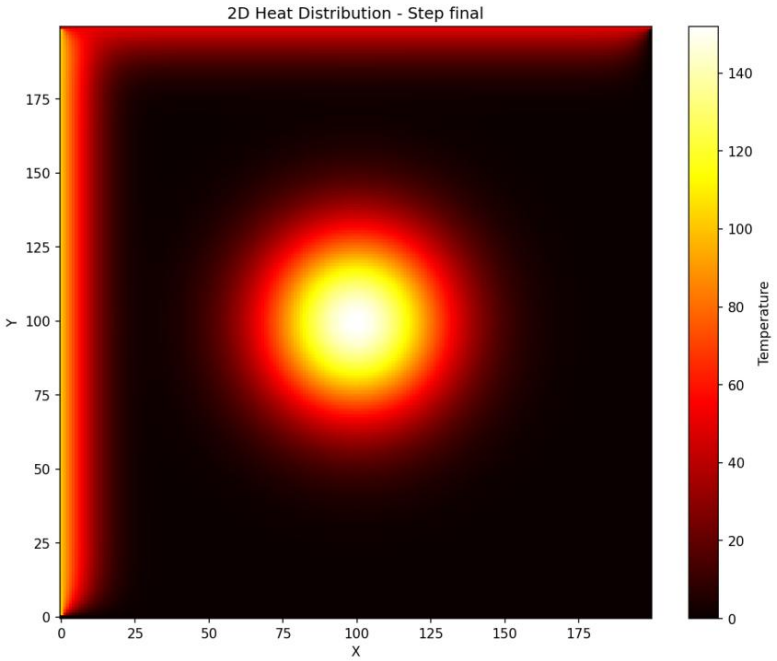

Figure 3- Final Temperature Distribution Using Blocking MPI Communication

This visualization confirms that heat diffuses normally across the full domain, even though the computation was split across multiple MPI processes.

4. Nonblocking Communication (MPI_Isend / MPI_Irecv)

To improve performance, a second version uses nonblocking communication. Here, processes initiate send and receive operations but do not wait for them to complete. Instead, they immediately begin computing the interior points, which do not depend on halo cells.

Only when updating boundary points do processes call **MPI_Waitall()** to ensure all halo data has been received.

This approach allows communication and computation to overlap**,** reducing idle time and improving scalability.

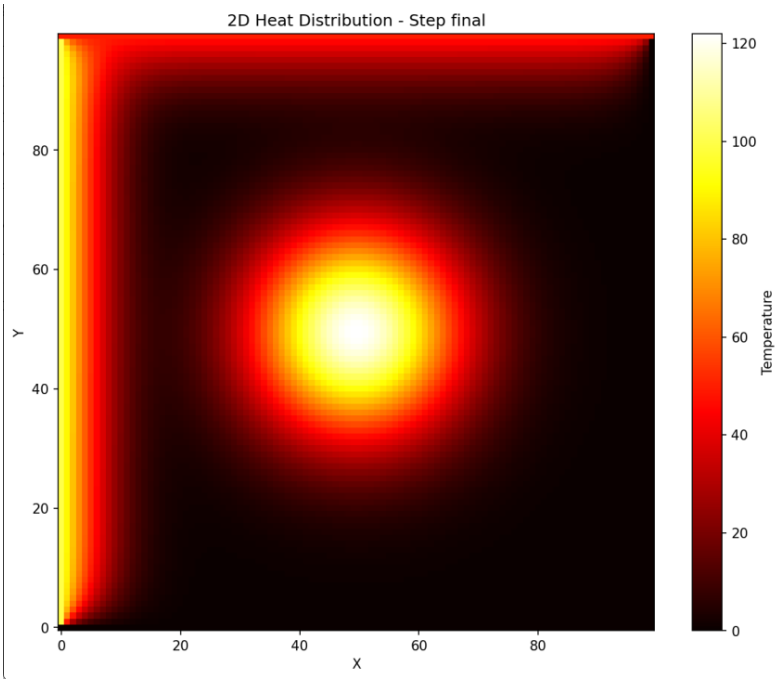

Figure 4- Final Temperature Distribution Using Nonblocking MPI Communication

5. Parallel Solver Correctness Verification

To ensure that parallelization did not introduce numerical errors, we compare the serial solver output to the MPI output using a visual verification script. The resulting figure shows near-identical patterns, confirming:

- correct implementation of halo exchange
- identical numerical updates
- accurate process synchronization

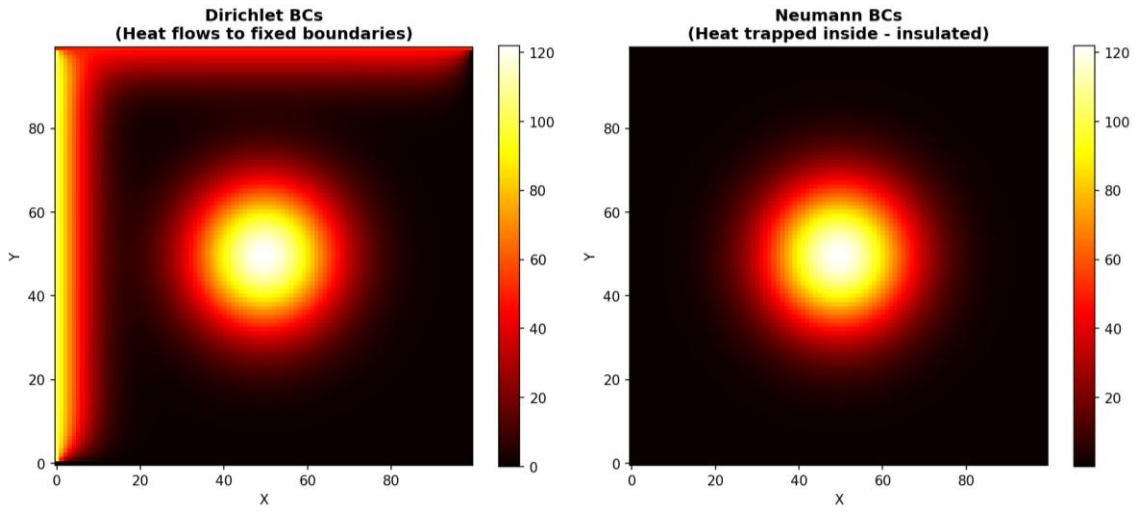

Figure 5- Serial vs Parallel Consistency Check

V. Performance Evaluation

1. Strong Scaling Analysis

Strong scaling measures how execution time decreases when we increase the number of MPI processes while keeping the problem size fixed.

The figure shows that the speedup improves when moving from 1 to 4 processes, reaching a maximum near 4 processes. Beyond this point, the performance drops significantly.

This behavior is expected for relatively small grids (100×100), where communication overhead becomes dominant and limits performance for higher process counts.

The reduction in speedup for 8 and 16 processes indicates that the cost of halo exchanges outweighs the benefit of parallel computation at this scale.

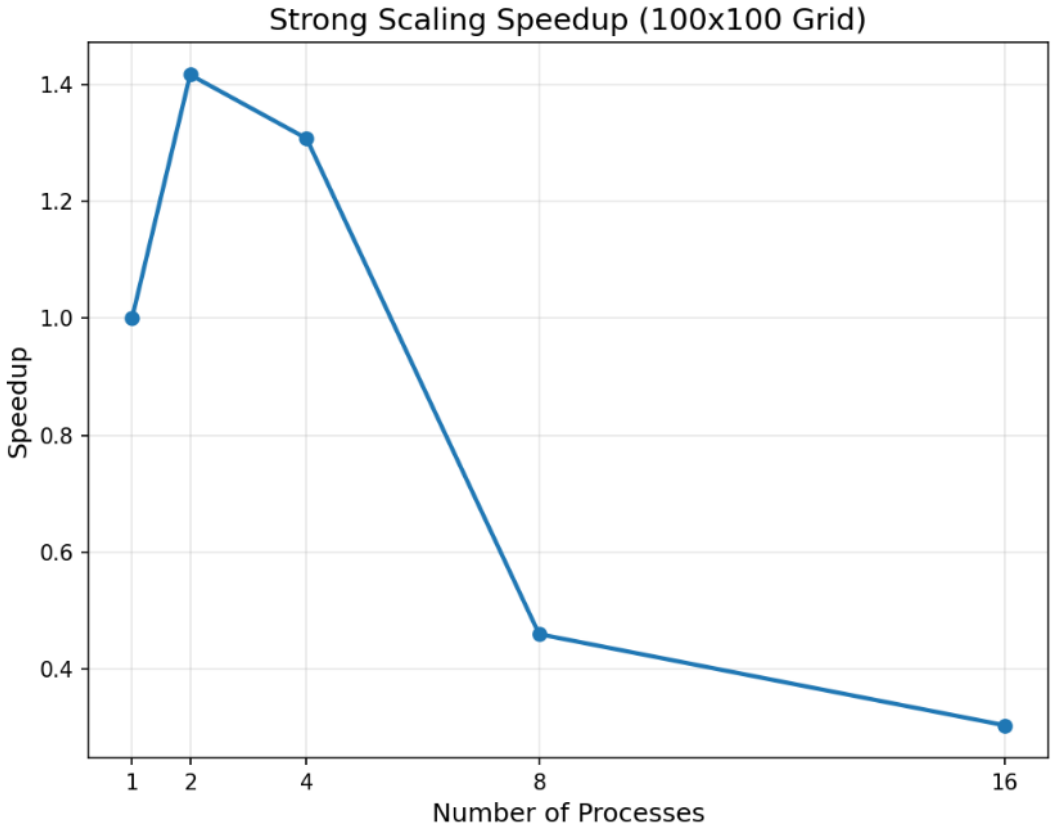

Figure 6- Strong scaling speedup

2. Weak Scaling Analysis

Weak scaling examines whether the solver maintains consistent performance when both the number of MPI processes and the problem size grow proportionally.
The weak scaling results indicate relatively stable execution time for smaller numbers of processes but show a gradual increase as more processes are added.
This reflects the additional communication required for halo exchanges and the increased cost of synchronizing more MPI ranks.
Although not perfectly flat, the weak scaling curve demonstrates acceptable performance for moderate process counts and confirms the solver's ability to handle larger distributed domains effectively.

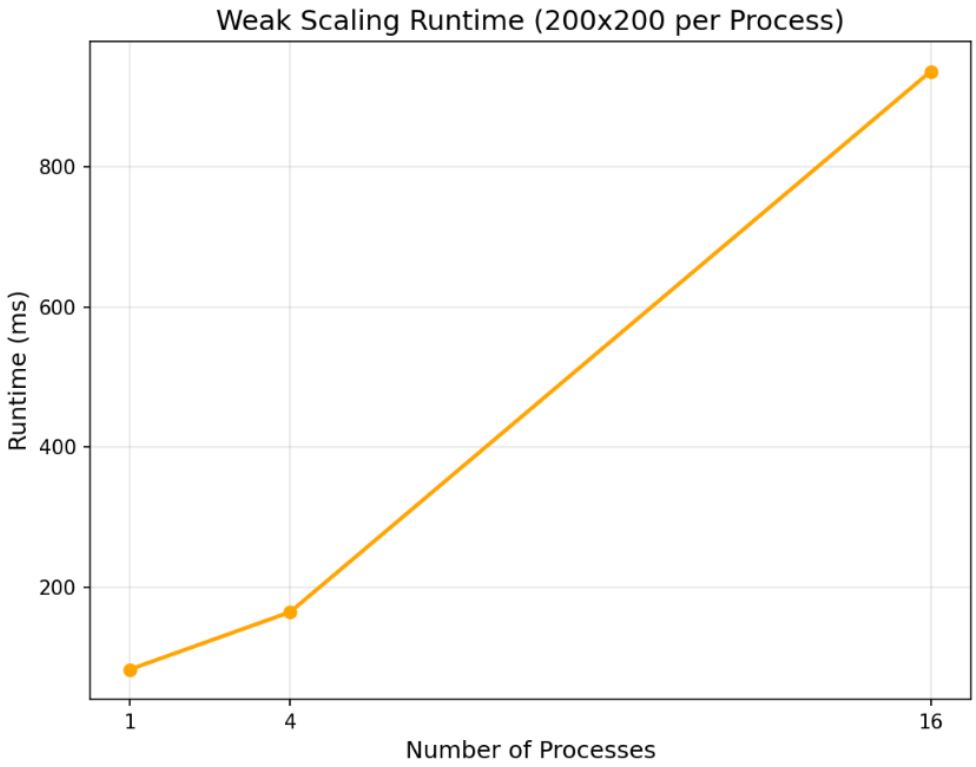

Figure 7- Weak scaling

3. Maximum Problem Size and Solver Limitations

During our experiments, we also increased the grid resolution to determine the maximum domain size the solver could handle on the available hardware. The largest configuration that successfully executed was $\boldsymbol{N_x = 5000}$ and $\boldsymbol{N_y = 5000}$, corresponding to 25 million grid points. Beyond this size, the simulation was limited by memory consumption and increased MPI communication overhead, which caused excessive execution times or insufficient available RAM. Therefore, the **5000×5000 grid** represents the practical upper limit of our implementation on the tested system. This result highlights the scalability benefits of the MPI solver while also showing the hardware constraints encountered when solving very large heat diffusion problems.

VI. Team Contributions

Lea Chamseddine – Master Node (Rank 0):

- Designed and implemented the core serial and parallel solver framework, including domain decomposition and MPI process management.
- Developed the master node logic, responsible for initializing the computation, coordinating communication between processes, and gathering results for output and visualization.
- Maintained build scripts and managed the GitHub repository to ensure that the consolidated version of the project was pushed online.

Fatima Al Rabih – Worker Node 1 (Rank 1):

- Developed the numerical methods for the solver, including the finite-difference discretization and stability analysis for the 2D heat equation.
- Implemented and tested boundary conditions (Dirichlet and Neumann) for her subdomain.
- Assisted in debugging inter-process communication and validating the correctness of the computed temperature fields.
- Contributed to documenting the numerical methods, boundary conditions, and derivation of the solver in the technical report.

Kawsar Zbib – Worker Node 2 (Rank 2):

- Implemented the update kernels for interior and boundary cells, ensuring correct computations across the subdomains.
- Assisted in integrating non-blocking MPI communication and tested halo/ghost cell exchanges for stability and correctness.
- Conducted performance experiments, including strong and weak scaling tests, and contributed to analyzing the results.
- Helped validate the numerical outputs and contributed to sections of the technical report related to performance evaluation.

Christina Haber – Worker Node 3 (Rank 3):

- Collaborated with Kawsar on numerical updates, particularly boundary cell updates after halo communication.
- Assisted in debugging and validating numerical stability and correctness across different process configurations.
- Developed visualization scripts and tools to display temperature fields, heat maps, and scaling results.
- Helped analyze and interpret results for the report, including performance and visualization outcomes.

GitHub and Project Integration:

- All team members worked collaboratively on a shared virtual machine, transferring files between nodes via SCP.

• Only the master node (Lea) pushed the final consolidated project to GitHub.
• The repository includes the solver code, visualization scripts, build scripts, performance data, and analysis results.

VII. Conclusion

This project demonstrated the implementation and parallelization of a 2D heat diffusion solver using finite difference methods and MPI. We first developed a serial explicit solver, validated its results, and applied two types of boundary conditions (Dirichlet and Neumann). The MPI extension introduced domain decomposition and halo exchanges, enabling the solver to run efficiently on multiple processes using both blocking and nonblocking communication.

Performance analysis showed that the solver achieves reasonable speedup for small to moderate process counts. Strong scaling revealed a peak speedup around 4 processes due to communication overhead on small grids, while weak scaling indicated acceptable efficiency as the problem size increased proportionally with the number of processes. The solver was also tested on large grids, and the maximum feasible domain size was found to be 5000×5000 nodes, limited by available memory and communication cost.

Overall, this work highlights the benefits and challenges of parallelizing numerical simulations. The project provided practical experience in MPI programming, numerical methods, performance tuning, and scientific visualization skills essential for computational engineering and high-performance computing.